

UI navigation controller

Article • 10/20/2022

This page describes the basics of programming for UI navigation devices using [Windows.Gaming.Input.UINavigationController](#) and related APIs for the Universal Windows Platform (UWP).

By reading this page, you'll learn:

- How to gather a list of connected UI navigation devices and their users
- How to detect that a navigation device has been added or removed
- How to read input from one or more UI navigation devices
- How gamepads and arcade sticks behave as navigation devices

UI navigation controller overview

Almost all games have at least some user interface that's separate from gameplay, even if it's just pregame menus or in-game dialogs. Players need to be able to navigate this UI using whichever input device they've chosen, but it burdens developers to add specific support for each kind of input device and can also introduce inconsistencies between games and input devices that confuse players. For these reasons the [UINavigationController](#) API was created.

UI navigation controllers are *logical* input devices that exist to provide a vocabulary of common UI Navigation commands that can be supported by a variety of *physical* input devices. A *UI navigation controller* is just a different way of looking at a physical input device; we use *navigation device* to refer to any physical input device being viewed as a navigation controller. By programming against a navigation device rather than specific input devices, developers avoid the burden of supporting different input devices and achieve consistency by default.

Because the number and variety of controls supported by each kind of input device can be so different and because certain input devices might want to support a richer set of navigation commands, the navigation controller interface divides the vocabulary of commands into a *required set* containing the most common and essential commands, and an *optional set* containing convenient but nonessential commands. All navigation devices support every command in the *required set* and may support all, some, or none of the commands in the *optional set*.

Required set

Navigation devices must support all navigation commands in the *required set*; these are the directional (up, down, left, and right), view, menu, accept, and cancel commands.

The directional commands are intended for primary [XY-focus navigation](#) between single UI elements. The view and menu commands are intended for displaying gameplay information (often momentary, sometimes modally) and for switching between gameplay and menu contexts, respectively. The accept and cancel commands are intended for affirmative (yes) and negative (no) responses, respectively.

The following table summarizes these commands and their intended uses, with examples.

Command	Intended use
Up	XY-focus navigation up
Down	XY-focus navigation down
Left	XY-focus navigation left
Right	XY-focus navigation right
View	Display gameplay info (<i>scoreboard, game stats, objectives, world or area map</i>)
Menu	Primary menu / Pause (<i>settings, status, equipment, inventory, pause</i>)
Accept	Affirmative response (<i>accept, advance, confirm, start, yes</i>)
Cancel	Negative response (<i>reject, reverse, decline, stop, no</i>)

Optional set

Navigation devices may also support all, some, or none of the navigation commands in the *optional set*; these are the paging (up, down, left, and right), scrolling (up, down, left, and right), and contextual (context 1-4) commands.

The contextual commands are explicitly intended for application-specific commands and navigation shortcuts. The paging and scrolling commands are intended for quick navigation between pages or groups of UI elements and for fine-grained navigation within UI elements, respectively.

The following table summarizes these commands and their intended uses.

Command	Intended use
PageUp	Jump upward (to upper/previous vertical page or group)
PageDown	Jump downward (to lower/next vertical page or group)
PageLeft	Jump left (to leftward/previous horizontal page or group)
PageRight	Jump right (to rightward/next horizontal page or group)
ScrollUp	Scroll up (within focused UI element or scrollable group)
ScrollDown	Scroll down (within focused UI element or scrollable group)
ScrollLeft	Scroll left (within focused UI element or scrollable group)
ScrollRight	Scroll right (within focused UI element or scrollable group)
Context1	Primary context action
Context2	Secondary context action
Context3	Third context action
Context4	Fourth context action

Note Although a game is free to respond to any command with an actual function that is different than its intended use, surprising behavior should be avoided. In particular, don't change the actual function of a command if you need its intended

use, try to assign novel functions to the command that makes the most sense, and assign counterpart functions to counterpart commands such as PageUp/PageDown. Finally, consider which commands are supported by each kind of input device and which controls they are mapped to, making sure that critical commands are accessible from every device.

Gamepad, arcade stick, and racing wheel navigation

All input devices supported by the Windows.Gaming.Input namespace are UI navigation devices.

The following table summarizes how the *required* set of navigation commands map to various input devices.

Navigation command	Gamepad input	Arcade stick input	Racing Wheel input
Up	Left thumbstick up / D-pad up	Stick up	D-pad up
Down	Left thumbstick down / D-pad down	Stick down	D-pad down
Left	Left thumbstick left / D-pad left	Stick left	D-pad left
Right	Left thumbstick right / D-pad right	Stick right	D-pad right
View	View button	View button	View button
Menu	Menu button	Menu button	Menu button
Accept	A button	Action 1 button	A button
Cancel	B button	Action 2 button	B button

The following table summarizes how the *optional* set of navigation commands map to various input devices.

Navigation command	Gamepad input	Arcade stick input	Racing Wheel input
PageUp	Left trigger	<i>not supported</i>	<i>varies</i>
PageDown	Right trigger	<i>not supported</i>	<i>varies</i>
PageLeft	Left bumper	<i>not supported</i>	<i>varies</i>

Navigation command	Gamepad input	Arcade stick input	Racing Wheel input
PageRight	Right bumper	<i>not supported</i>	<i>varies</i>
ScrollUp	Right thumbstick up	<i>not supported</i>	<i>varies</i>
ScrollDown	Right thumbstick down	<i>not supported</i>	<i>varies</i>
ScrollLeft	Right thumbstick left	<i>not supported</i>	<i>varies</i>
ScrollRight	Right thumbstick right	<i>not supported</i>	<i>varies</i>
Context1	X button	<i>not supported</i>	X button (<i>commonly</i>)
Context2	Y button	<i>not supported</i>	Y button (<i>commonly</i>)
Context3	Left thumbstick press	<i>not supported</i>	<i>varies</i>
Context4	Right thumbstick press	<i>not supported</i>	<i>varies</i>

Detect and track UI navigation controllers

Although UI navigation controllers are logical input devices, they are a representation of a physical device and are managed by the system in the same way. You don't have to create or initialize them; the system provides a list of connected UI navigation controllers and events to notify you when a UI Navigation controller is added or removed.

The UI navigation controllers list

The [UINavigationController](#) class provides a static property, [UINavigationControllerControllers](#), which is a read-only list of UI navigation devices that are currently connected. Because you might only be interested in some of the connected navigation devices, its recommended that you maintain your own collection instead of accessing them through the `UINavigationControllerControllers` property.

The following example copies all connected UI navigation controllers into a new collection.

C++

```
auto myNavigationControllers = ref new Vector<UINavigationController^>();

for (auto device : UINavigationController::UINavigationControllerControllers)
{
    // This code assumes that you're interested in all navigation
```

```
controllers.  
    myNavigationControllers->Append(device);  
}
```

Adding and removing UI navigation controllers

When a UI navigation controller is added or removed the [UINavigationControllerAdded](#) and [UINavigationControllerRemoved](#) events are raised. You can register an event handler for these events to keep track of the navigation devices that are currently connected.

The following example starts tracking a UI navigation device that's been added.

C++

```
UINavigationController::UINavigationControllerAdded += ref new  
EventHandler<UINavigationController^>(Platform::Object^,  
UINavigationController^ args)  
{  
    // This code assumes that you're interested in all new navigation  
    controllers.  
    myNavigationControllers->Append(args);  
}
```

The following example stops tracking an arcade stick that's been removed.

C++

```
UINavigationController::UINavigationControllerRemoved += ref new  
EventHandler<UINavigationController^>(Platform::Object^,  
UINavigationController^ args)  
{  
    unsigned int indexRemoved;  
  
    if(myNavigationControllers->IndexOf(args, &indexRemoved))  
    {  
        myNavigationControllers->RemoveAt(indexRemoved);  
    }  
}
```

Users and headsets

Each navigation device can be associated with a user account to link their identity to their input, and can have a headset attached to facilitate voice chat or navigation features. To learn more about working with users and headsets, see [Tracking users and their devices](#) and [Headset](#).

Reading the UI navigation controller

After you identify the UI navigation device that you're interested in, you're ready to gather input from it. However, unlike some other kinds of input that you might be used to, navigation devices don't communicate state-change by raising events. Instead, you take regular readings of their current state by *polling* them.

Polling the UI navigation controller

Polling captures a snapshot of the navigation device at a precise point in time. This approach to input gathering is a good fit for most games because their logic typically runs in a deterministic loop rather than being event-driven; its also typically simpler to interpret game commands from input gathered all at once than it is from many single inputs gathered over time.

You poll a navigation device by calling `UINavigationController.GetCurrentReading`; this function returns a `UINavigationControllerReading` that contains the state of the navigation device.

C++

```
auto navigationController = myNavigationControllers[0];

UINavigationControllerReading reading = navigationController->GetCurrentReading();
```

Reading the buttons

Each of the UI navigation buttons provide a boolean reading that corresponds to whether its pressed (down), or released (up). For efficiency, button readings aren't represented as individual boolean values; instead they're all packed into one of two bitfields represented by the `RequiredUINavigationControllerButtons` and `OptionalUINavigationControllerButtons` enumerations.

The buttons belonging to the *required set* are read from the `RequiredButtons` property of the `UINavigationControllerReading` structure; the buttons belonging to the *optional set* are read from the `OptionalButtons` property. Because these properties are bitfields, bitwise masking is used to isolate the value of the button that you're interested in. The button is pressed (down) when the corresponding bit is set; otherwise its released (up).

The following example determines whether the Accept button in the *required set* is pressed.

C++

```
if (RequiredUINavigationButtons::Accept == (reading.RequiredButtons &
RequiredUINavigationButtons::Accept))
{
    // Accept is pressed
}
```

The following example determines whether the Accept button in the *required set* is released.

C++

```
if (RequiredUINavigationButtons::None == (reading.RequiredButtons &
RequiredUINavigationButtons::Accept))
{
    // Accept is released (not pressed)
}
```

Be sure to use the `OptionalButtons` property and `OptionalUINavigationButtons` enumeration when reading buttons in the *optional set*.

The following example determines whether the Context 1 button in the *optional set* is pressed.

C++

```
if (OptionalUINavigationButtons::Context1 == (reading.OptionalButtons &
OptionalUINavigationButtons::Context1))
{
    // Context 1 is pressed
}
```

Sometimes you might want to determine when a button transitions from pressed to released or released to pressed, whether multiple buttons are pressed or released, or if a set of buttons are arranged in a particular way--some pressed, some not. For information on how to detect these conditions, see [Detecting button transitions](#) and [Detecting complex button arrangements](#).

Run the UI navigation controller sample

The [InputInterfacingUWP sample \(github\)](#) demonstrates how the different input devices behave as UI navigation controllers.

See also

Windows.Gaming.Input.Gamepad Windows.Gaming.Input.ArcadeStick

Windows.Gaming.Input.RacingWheel Windows.Gaming.Input.IGameController

Input practices for games

Article • 05/02/2023

This topic describes patterns and techniques for effectively using input devices in Universal Windows Platform (UWP) games.

By reading this topic, you'll learn:

- how to track players and which input and navigation devices they're currently using
- how to detect button transitions (pressed-to-released, released-to-pressed)
- how to detect complex button arrangements with a single test

Choosing an input device class

There are many different types of input APIs available to you, such as [ArcadeStick](#), [FlightStick](#), and [Gamepad](#). How do you decide which API to use for your game?

You should choose whichever API gives you the most appropriate input for your game. For example, if you're making a 2D platform game, you can probably just use the **Gamepad** class and not bother with the extra functionality available via other classes. This would restrict the game to supporting gamepads only and provide a consistent interface that will work across many different gamepads with no need for additional code.

On the other hand, for complex flight and racing simulations, you might want to enumerate all of the [RawGameController](#) objects as a baseline to make sure they support any niche device that enthusiast players might have, including devices such as separate pedals or throttle that are still used by a single player.

From there, you can use an input class's **FromGameController** method, such as [Gamepad.FromGameController](#), to see if each device has a more curated view. For example, if the device is also a **Gamepad**, then you might want to adjust the button mapping UI to reflect that, and provide some sensible default button mappings to choose from. (This is in contrast to requiring the player to manually configure the gamepad inputs if you're only using **RawGameController**.)

Alternatively, you can look at the vendor ID (VID) and product ID (PID) of a **RawGameController** (using [HardwareVendorId](#) and [HardwareProductId](#), respectively) and provide suggested button mappings for popular devices while still remaining

compatible with unknown devices that come out in the future via manual mappings by the player.

Keeping track of connected controllers

While each controller type includes a list of connected controllers (such as [Gamepad.Gamepads](#)), it is a good idea to maintain your own list of controllers. See [The gamepads list](#) for more information (each controller type has a similarly named section on its own topic).

However, what happens when the player unplugs their controller, or plugs in a new one? You need to handle these events, and update your list accordingly. See [Adding and removing gamepads](#) for more information (again, each controller type has a similarly named section on its own topic).

Because the added and removed events are raised asynchronously, you could get incorrect results when dealing with your list of controllers. Therefore, anytime you access your list of controllers, you should put a lock around it so that only one thread can access it at a time. This can be done with the [Concurrency Runtime](#), specifically the [critical_section class](#), in `<ppl.h>`.

Another thing to think about is that the list of connected controllers will initially be empty, and takes a second or two to populate. So if you only assign the current gamepad in the start method, it will be `null`!

To rectify this, you should have a method that "refreshes" the main gamepad (in a single-player game; multiplayer games will require more sophisticated solutions). You should then call this method in both your controller added and controller removed event handlers, or in your update method.

The following method simply returns the first gamepad in the list (or `nullptr` if the list is empty). Then you just need to remember to check for `nullptr` anytime you do anything with the controller. It's up to you whether you want to block gameplay when there's no controller connected (for example, by pausing the game) or simply have gameplay continue, while ignoring input.

C++

```
#include <ppl.h>

using namespace Platform::Collections;
using namespace Windows::Gaming::Input;
using namespace concurrency;
```

```

Vector<Gamepad^>^ m_myGamepads = ref new Vector<Gamepad^>();

Gamepad^ GetFirstGamepad()
{
    Gamepad^ gamepad = nullptr;
    critical_section::scoped_lock{ m_lock };

    if (m_myGamepads->Size > 0)
    {
        gamepad = m_myGamepads->GetAt(0);
    }

    return gamepad;
}

```

Putting it all together, here is an example of how to handle input from a gamepad:

C++

```

#include <algorithm>
#include <ppl.h>

using namespace Platform::Collections;
using namespace Windows::Foundation;
using namespace Windows::Gaming::Input;
using namespace concurrency;

static Vector<Gamepad^>^ m_myGamepads = ref new Vector<Gamepad^>();
static Gamepad^ m_gamepad = nullptr;
static critical_section m_lock{};

void Start()
{
    // Register for gamepad added and removed events.
    Gamepad::GamepadAdded += ref new EventHandler<Gamepad^>
(&OnGamepadAdded);
    Gamepad::GamepadRemoved += ref new EventHandler<Gamepad^>
(&OnGamepadRemoved);

    // Add connected gamepads to m_myGamepads.
    for (auto gamepad : Gamepad::Gamepads)
    {
        OnGamepadAdded(nullptr, gamepad);
    }
}

void Update()
{
    // Update the current gamepad if necessary.
    if (m_gamepad == nullptr)
    {
        auto gamepad = GetFirstGamepad();
    }
}

```

```

        if (m_gamepad != gamepad)
        {
            m_gamepad = gamepad;
        }
    }

    if (m_gamepad != nullptr)
    {
        // Gather gamepad reading.
    }
}

// Get the first gamepad in the list.
Gamepad^ GetFirstGamepad()
{
    Gamepad^ gamepad = nullptr;
    critical_section::scoped_lock{ m_lock };

    if (m_myGamepads->Size > 0)
    {
        gamepad = m_myGamepads->GetAt(0);
    }

    return gamepad;
}

void OnGamepadAdded(Platform::Object^ sender, Gamepad^ args)
{
    // Check if the just-added gamepad is already in m_myGamepads; if it
    // isn't,
    // add it.
    critical_section::scoped_lock lock{ m_lock };
    auto it = std::find(begin(m_myGamepads), end(m_myGamepads), args);

    if (it == end(m_myGamepads))
    {
        m_myGamepads->Append(args);
    }
}

void OnGamepadRemoved(Platform::Object^ sender, Gamepad^ args)
{
    // Remove the gamepad that was just disconnected from m_myGamepads.
    unsigned int indexRemoved;
    critical_section::scoped_lock lock{ m_lock };

    if (m_myGamepads->IndexOf(args, &indexRemoved))
    {
        if (m_gamepad == m_myGamepads->GetAt(indexRemoved))
        {
            m_gamepad = nullptr;
        }

        m_myGamepads->RemoveAt(indexRemoved);
    }
}

```

```
}  
}
```

Tracking users and their devices

All input devices are associated with a [User](#) so that their identity can be linked to their gameplay, achievements, settings changes, and other activities. Users can sign in or sign out at will, and it's common for a different user to sign in on an input device that remains connected to the system after the previous user has signed out. When a user signs in or out, the [IGameController.UserChanged](#) event is raised. You can register an event handler for this event to keep track of players and the devices they're using.

User identity is also the way that an input device is associated with its corresponding [UI navigation controller](#).

For these reasons, player input should be tracked and correlated with the [User](#) property of the device class (inherited from the [IGameController](#) interface).

The [UserGamepadPairingUWP](#) [sample app](#) on GitHub demonstrates how you can keep track of users and the devices they're using.

Detecting button transitions

Sometimes you want to know when a button is first pressed or released; that is, precisely when the button state transitions from released to pressed or from pressed to released. To determine this, you need to remember the previous device reading and compare the current reading against it to see what's changed.

The following example demonstrates a basic approach for remembering the previous reading; gamepads are shown here, but the principles are the same for arcade stick, racing wheel, and the other input device types.

C++

```
Gamepad gamepad;  
GamepadReading newReading();  
GamepadReading oldReading();  
  
// Called at the start of the game.  
void Game::Start()  
{  
    gamepad = Gamepad::Gamepads[0];  
}
```

```
// Game::Loop represents one iteration of a typical game loop
void Game::Loop()
{
    // move previous newReading into oldReading before getting next
    newReading
    oldReading = newReading, newReading = gamepad.GetCurrentReading();

    // process device readings using buttonJustPressed/buttonJustReleased
    (see below)
}
```

Before doing anything else, `Game::Loop` moves the existing value of `newReading` (the gamepad reading from the previous loop iteration) into `oldReading`, then fills `newReading` with a fresh gamepad reading for the current iteration. This gives you the information you need to detect button transitions.

The following example demonstrates a basic approach for detecting button transitions:

C++

```
bool ButtonJustPressed(const GamepadButtons selection)
{
    bool newSelectionPressed = (selection == (newReading.Buttons &
selection));
    bool oldSelectionPressed = (selection == (oldReading.Buttons &
selection));

    return newSelectionPressed && !oldSelectionPressed;
}

bool ButtonJustReleased(GamepadButtons selection)
{
    bool newSelectionReleased =
        (GamepadButtons.None == (newReading.Buttons & selection));

    bool oldSelectionReleased =
        (GamepadButtons.None == (oldReading.Buttons & selection));

    return newSelectionReleased && !oldSelectionReleased;
}
```

These two functions first derive the Boolean state of the button selection from `newReading` and `oldReading`, then perform Boolean logic to determine whether the target transition has occurred. These functions return **true** only if the new reading contains the target state (pressed or released, respectively) *and* the old reading does not also contain the target state; otherwise, they return **false**.

Detecting complex button arrangements

Each button of an input device provides a digital reading that indicates whether it's pressed (down) or released (up). For efficiency, button readings aren't represented as individual boolean values; instead, they're all packed into bitfields represented by device-specific enumerations such as [GamepadButtons](#). To read specific buttons, bitwise masking is used to isolate the values that you're interested in. A button is pressed (down) when its corresponding bit is set; otherwise, it's released (up).

Recall how single buttons are determined to be pressed or released; gamepads are shown here, but the principles are the same for arcade stick, racing wheel, and the other input device types.

C++

```
GamepadReading reading = gamepad.GetCurrentReading();

// Determines whether gamepad button A is pressed.
if (GamepadButtons::A == (reading.Buttons & GamepadButtons::A))
{
    // The A button is pressed.
}

// Determines whether gamepad button A is released.
if (GamepadButtons::None == (reading.Buttons & GamepadButtons::A))
{
    // The A button is released (not pressed).
}
```

As you can see, determining the state of a single button is straight-forward, but sometimes you might want to determine whether multiple buttons are pressed or released, or if a set of buttons are arranged in a particular way—some pressed, some not. Testing multiple buttons is more complex than testing single buttons—especially with the potential of mixed button state—but there's a simple formula for these tests that applies to single and multiple button tests alike.

The following example determines whether gamepad buttons A and B are both pressed:

C++

```
if ((GamepadButtons::A | GamepadButtons::B) == (reading.Buttons &
(GamepadButtons::A | GamepadButtons::B))
{
    // The A and B buttons are both pressed.
}
```

The following example determines whether gamepad buttons A and B are both released:

C++

```
if ((GamepadButtons::None == (reading.Buttons & GamepadButtons::A |
GamepadButtons::B))
{
    // The A and B buttons are both released (not pressed).
}
```

The following example determines whether gamepad button A is pressed while button B is released:

C++

```
if (GamepadButtons::A == (reading.Buttons & (GamepadButtons::A |
GamepadButtons::B))
{
    // The A button is pressed and the B button is released (B is not
    pressed).
}
```

The formula that all five of these examples have in common is that the arrangement of buttons to be tested for is specified by the expression on the left-hand side of the equality operator while the buttons to be considered are selected by the masking expression on the right-hand side.

The following example demonstrates this formula more clearly by rewriting the previous example:

C++

```
auto buttonArrangement = GamepadButtons::A;
auto buttonSelection = (reading.Buttons & (GamepadButtons::A |
GamepadButtons::B));

if (buttonArrangement == buttonSelection)
{
    // The A button is pressed and the B button is released (B is not
    pressed).
}
```

This formula can be applied to test any number of buttons in any arrangement of their states.

Get the state of the battery

For any game controller that implements the [IGameControllerBatteryInfo](#) interface, you can call [TryGetBatteryReport](#) on the controller instance to get a [BatteryReport](#) object that provides information about the battery in the controller. You can get properties like the rate that the battery is charging ([ChargeRateInMilliwatts](#)), the estimated energy capacity of a new battery ([DesignCapacityInMilliwattHours](#)), and the fully-charged energy capacity of the current battery ([FullChargeCapacityInMilliwattHours](#)).

For game controllers that support detailed battery reporting, you can get this and more information about the battery, as detailed in [Get battery information](#). However, most game controllers don't support that level of battery reporting, and instead use low-cost hardware. For these controllers, you'll need to keep the following considerations in mind:

- **ChargeRateInMilliwatts** and **DesignCapacityInMilliwattHours** will always be **NULL**.
- You can get the battery percentage by calculating [RemainingCapacityInMilliwattHours](#) / [FullChargeCapacityInMilliwattHours](#). You should ignore the values of these properties and only deal with the calculated percentage.
- The percentage from the previous bullet point will always be one of the following:
 - 100% (Full)
 - 70% (Medium)
 - 40% (Low)
 - 10% (Critical)

If your code performs some action (like drawing UI) based on the percentage of battery life remaining, make sure that it conforms to the values above. For example, if you want to warn the player when the controller's battery is low, do so when it reaches 10%.

See also

- [Windows.System.User](#) class
- [Windows.Gaming.Input.IGameController](#) interface
- [Windows.Gaming.Input.GamepadButtons](#) enum
- [UserGamepadPairingUWP](#) sample app [↗](#)

Registry data for game controllers

Article • 10/20/2022

ⓘ Note

This topic is meant for manufacturers of Windows 10-compatible game controllers, and doesn't apply to the majority of developers.

The [Windows.Gaming.Input namespace](#) allows independent hardware vendors (IHVs) to add data to the PC's registry, enabling their devices to appear as [Gamepads](#), [RacingWheels](#), [ArcadeSticks](#), [FlightSticks](#), and [UINavigationControllers](#) as appropriate. All IHVs should add this data for their compatible controllers. By doing this, all UWP games (and any desktop games that use the WinRT API) will be able to support your game controller.

Mapping scheme

Mappings for a device with Vendor ID (VID) **VVVV**, Product ID (PID) **PPPP**, Usage Page **UUUU**, and Usage ID **XXXX**, will be read out from this location in the registry:

```
HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\GameInput\Devices\VVVVPPPPUUUU  
XXXX
```

The table below explains the expected values under the device root location:

Name	Type	Required?	Info
Disabled	DWORD	No	Indicates that this particular device should be disabled. 0: Device is not disabled. 1: Device is disabled.
Description	REG_SZ	No	A short description of the device.

Your device installer should add this data to the registry (either via setup or an [INF file](#)).

Subkeys under the device root location are detailed in the following sections.

Gamepad

The table below lists the required and optional subkeys under the **Gamepad** subkey:

Subkey	Required?	Info
Menu	Yes	See Button mapping
View	Yes	
A	Yes	
B	Yes	
X	Yes	
Y	Yes	
LeftShoulder	Yes	
RightShoulder	Yes	
LeftThumbstickButton	Yes	
RightThumbstickButton	Yes	
DPadUp	Yes	
DPadDown	Yes	
DPadLeft	Yes	
DPadRight	Yes	
Paddle1	No	
Paddle2	No	
Paddle3	No	
Paddle4	No	
LeftTrigger	Yes	See Axis mapping
RightTrigger	Yes	
LeftThumbstickX	Yes	
LeftThumbstickY	Yes	
RightThumbstickX	Yes	
RightThumbstickY	Yes	

📌 **Note**

If you add your game controller as a supported **Gamepad**, we highly recommend that you also add it as a supported **UINavigationController**.

RacingWheel

The table below lists the required and optional subkeys under the **RacingWheel** subkey:

Subkey	Required?	Info
PreviousGear	Yes	See Button mapping
NextGear	Yes	
DPadUp	No	
DPadDown	No	
DPadLeft	No	
DPadRight	No	
Button1	No	
Button2	No	
Button3	No	
Button4	No	
Button5	No	
Button6	No	
Button7	No	
Button8	No	
Button9	No	
Button10	No	
Button11	No	
Button12	No	
Button13	No	
Button14	No	
Button15	No	

Button16	No	
FirstGear	No	
SecondGear	No	
ThirdGear	No	
FourthGear	No	
FifthGear	No	
SixthGear	No	
SeventhGear	No	
ReverseGear	No	
Wheel	Yes	See Axis mapping
Throttle	Yes	
Brake	Yes	
Clutch	No	
Handbrake	No	
MaxWheelAngle	Yes	See Properties mapping

ArcadeStick

The table below lists the required and optional subkeys under the **ArcadeStick** subkey:

Subkey	Required?	Info
Action 1	Yes	See Button mapping
Action2	Yes	
Action3	Yes	
Action4	Yes	
Action5	Yes	
Action6	Yes	
Special1	Yes	
Special2	Yes	

StickUp	Yes
StickDown	Yes
StickLeft	Yes
StickRight	Yes

FlightStick

The table below lists the required and optional subkeys under the **FlightStick** subkey:

Subkey	Required?	Info
FirePrimary	Yes	See Button mapping
FireSecondary	Yes	
Roll	Yes	See Axis mapping
Pitch	Yes	
Yaw	Yes	
Throttle	Yes	
HatSwitch	Yes	See Switch mapping

UINavigation

The table below lists the required and optional subkeys under **UINavigation** subkey:

Subkey	Required?	Info
Menu	Yes	See Button mapping
View	Yes	
Accept	Yes	
Cancel	Yes	
PrimaryUp	Yes	
PrimaryDown	Yes	
PrimaryLeft	Yes	
PrimaryRight	Yes	

Context1	No
Context2	No
Context3	No
Context4	No
PageUp	No
PageDown	No
PageLeft	No
PageRight	No
ScrollUp	No
ScrollDown	No
ScrollLeft	No
ScrollRight	No
SecondaryUp	No
SecondaryDown	No
SecondaryLeft	No
SecondaryRight	No

For more information about UI navigation controllers and the above commands, see [UI navigation controller](#).

Keys

The following sections explain the contents of each of the subkeys under the **Gamepad**, **RacingWheel**, **ArcadeStick**, **FlightStick**, and **UINavigation** keys.

Button mapping

The table below lists the values that are needed to map a button. For example, if pressing **DPadUp** on the game controller, the mapping for **DPadUp** should contain the **ButtonIndex** value (**Source** is **Button**). If **DPadUp** needs to be mapped from a switch position, then the **DPadUp** mapping should contain the values **SwitchIndex** and **SwitchPosition** (**Source** is **Switch**).

Source	Value name	Value type	Required?	Value info
Button	ButtonIndex	DWORD	Yes	Index in the RawGameController button array.
Axis	AxisIndex	DWORD	Yes	Index in the RawGameController axis array.
	Invert	DWORD	No	Indicates that the axis value should be inverted before the Threshold Percent and DebouncePercent factors are applied.
	ThresholdPercent	DWORD	Yes	Indicates the axis position at which the mapped button value transitions between the pressed and released states. The valid range of values is 0 to 100. The button is considered pressed if the axis value is greater than or equal to this value.
	DebouncePercent	DWORD	Yes	Defines the size of a window around the ThresholdPercent value, which is used to debounce the reported button state. The valid range of values is 0 to 100. Button state transitions can only occur when the axis value crosses the upper or lower boundaries of the debounce window. For example, a ThresholdPercent of 50 and DebouncePercent of 10 results in debounce boundaries at 45% and 55% of the full-range axis values. The button can't transition to the pressed state until the axis value reaches 55% or above, and it can't transition back to the released state until the axis value reaches 45% or below. The computed debounce window boundaries are clamped between 0% and 100%. For example, a threshold of 5% and a debounce window of 20% would result in the debounce window boundaries falling at 0% and 15%. The button state for axis values of 0% and 100% are always reported as released and pressed, respectively, regardless of the threshold and debounce values.
Switch	SwitchIndex	DWORD	Yes	Index in the RawGameController switch array.
	SwitchPosition	REG_SZ	Yes	Indicates the switch position that will cause the mapped button to report that it's being

pressed. The position values can be one of these strings:

- Up
- UpRight
- Right
- DownRight
- Down
- DownLeft
- Left
- UpLeft

IncludeAdjacent	DWORD	No	Indicates that adjacent switch positions will also cause the mapped button to report that it's being pressed.
-----------------	-------	----	---

Axis mapping

The table below lists the values that are needed to map an axis:

Source	Value name	Value type	Required?	Value info		
Button	MaxValueButtonIndex	DWORD	Yes	Index in the RawGameController button array which gets translated to the mapped unidirectional axis value.		
				MaxButton	AxisValue	
				FALSE	0.0	
				TRUE	1.0	
	MinValueButtonIndex	DWORD	No	Indicates that the mapped axis is bidirectional. Values of MaxButton and MinButton are combined into a single bidirectional axis as shown below.		
				MinButton	MaxButton	AxisValue
				FALSE	FALSE	0.5
				FALSE	TRUE	1.0
				TRUE	FALSE	0.0
				TRUE	TRUE	0.5

Axis	AxisIndex	DWORD	Yes	Index in the RawGameController axis array.										
	Invert	DWORD	No	Indicates that the mapped axis value should be inverted before it's returned.										
Switch	SwitchIndex	DWORD	Yes	Index in the RawGameController switch array.										
	MaxValueSwitchPosition	REG_SZ	Yes	One of the following strings: - Up - UpRight - Right - DownRight - Down - DownLeft - Left - UpLeft It indicates the position of the switch that causes the mapped axis value to be reported as 1.0. The opposing direction of MaxValueSwitchPosition is treated as 0.0. For example, if MaxValueSwitchPosition is Up, the axis value translation is shown below: <table><tr><th>Switch position</th><th>AxisValue</th></tr><tr><td>Up</td><td>1.0</td></tr><tr><td>Center</td><td>0.5</td></tr><tr><td>Down</td><td>0.0</td></tr></table>	Switch position	AxisValue	Up	1.0	Center	0.5	Down	0.0		
Switch position	AxisValue													
Up	1.0													
Center	0.5													
Down	0.0													
	IncludeAdjacent	DWORD	No	Indicates that adjacent switch positions will also cause the mapped axis to report 1.0. In the above example, if IncludeAdjacent is set, then the axis translation is done as follows: <table><tr><th>Switch position</th><th>AxisValue</th></tr><tr><td>Up</td><td>1.0</td></tr><tr><td>UpRight</td><td>1.0</td></tr><tr><td>UpLeft</td><td>1.0</td></tr><tr><td>Center</td><td>0.5</td></tr></table>	Switch position	AxisValue	Up	1.0	UpRight	1.0	UpLeft	1.0	Center	0.5
Switch position	AxisValue													
Up	1.0													
UpRight	1.0													
UpLeft	1.0													
Center	0.5													

	Down	0.0
	DownRight	0.0
	DownLeft	0.0

Switch mapping

Switch positions can be mapped either from a set of buttons in the buttons array of the **RawGameController** or from an index in the switches array. Switch positions can't be mapped from axes.

Source	Value name	Value type	Value info
Button	ButtonCount	DWORD	2, 4, or 8
	SwitchKind	REG_SZ	TwoWay , FourWay , or EightWay
	UpButtonIndex	DWORD	See *ButtonIndex values
	DownButtonIndex	DWORD	
	LeftButtonIndex	DWORD	
	RightButtonIndex	DWORD	
	UpRightButtonIndex	DWORD	
	DownRightButtonIndex	DWORD	
	DownLeftButtonIndex	DWORD	
	UpLeftButtonIndex	DWORD	
Axis	SwitchKind	REG_SZ	TwoWay , FourWay , or EightWay
	XAxisIndex	DWORD	YAxisIndex is always present. XAxisIndex is only present when SwitchKind is FourWay or EightWay .
	YAxisIndex	DWORD	
	XDeadZonePercent	DWORD	Indicate the size of the dead zone around the center position of the axes.
	YDeadZonePercent	DWORD	
	XDebouncePercent	DWORD	Define the size of the windows around the upper and lower dead zone limits, which are used to debounce the reported switch state.
	YDebouncePercent	DWORD	
	XInvert	DWORD	Indicate that the corresponding axis values should be inverted before the dead zone and debounce

	YInvert	DWORD	window calculations are applied.
Switch	SwitchIndex	DWORD	Index in the RawGameController switch array.
	Invert	DWORD	Indicates that the switch reports its positions in a counter-clockwise order, rather than the default clockwise order.
	PositionBias	DWORD	Shifts the starting point of how positions are reported by the specified amount. PositionBias is always counted clockwise from the original starting point, and is applied before the order of values is reversed. For example, a switch that reports positions starting with DownRight in counter-clockwise order can be normalized by setting the Invert flag and specifying a PositionBias of 5:

Position	Reported value	After PositionBias and Invert flags
DownRight	0	3
Right	1	2
UpRight	2	1
Up	3	0
UpLeft	4	7
Left	5	6
DownLeft	6	5
Down	7	4

*ButtonIndex values

*ButtonIndex values index into the **RawGameController**'s button array:

ButtonCount	SwitchKind	RequiredMappings
2	TwoWay	- UpButtonIndex - DownButtonIndex
4	FourWay	- UpButtonIndex - DownButtonIndex

		• LeftButtonIndex • RightButtonIndex
4	EightWay	• UpButtonIndex • DownButtonIndex • LeftButtonIndex • RightButtonIndex
8	EightWay	• UpButtonIndex • DownButtonIndex • LeftButtonIndex • RightButtonIndex • UpRightButtonIndex • DownRightButtonIndex • DownLeftButtonIndex • UpLeftButtonIndex

Properties mapping

These are static mapping values for different mapping types.

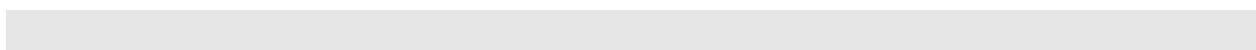
Mapping	Value name	Value type	Value info
RacingWheel	MaxWheelAngle	DWORD	Indicates the maximum physical wheel angle supported by the wheel in a single direction. For example, a wheel with a possible rotation of -90 degrees to 90 degrees would specify 90.

Labels

Labels should be present under the **Labels** key under the device root. **Labels** can have 3 subkeys: **Buttons**, **Axes**, and **Switches**.

Button labels

The **Buttons** key maps each of the button positions in the **RawGameController**'s buttons array to a string. Each string is mapped internally to the corresponding [GameControllerButtonLabel](#) enum value. For example, if a gamepad has ten buttons and the order in which the **RawGameController** parses out the buttons and presents them in the buttons report is like this:



C++

```
Menu,           // Index 0
View,           // Index 1
LeftStickButton, // Index 2
RightStickButton, // Index 3
LetterA,        // Index 4
LetterB,        // Index 5
LetterX,        // Index 6
LetterY,        // Index 7
LeftBumper,     // Index 8
RightBumper     // Index 9
```

The labels should appear in this order under the **Buttons** key:

Name	Value (type: REG_SZ)
Button0	Menu
Button1	View
Button2	LeftStickButton
Button3	RightStickButton
Button4	LetterA
Button5	LetterB
Button6	LetterX
Button7	LetterY
Button8	LeftBumper
Button9	RightBumper

Axis labels

The **Axes** key will map each of the axis positions in the **RawGameController**'s axis array to one of the labels listed in [GameControllerButtonLabel Enum](#) just like the button labels. See the example in [Button labels](#).

Switch labels

The **Switches** key maps switch positions to labels. The values follow this naming convention: to label a position of a switch, whose index is *x* in the **RawGameController**'s switch array, add these values under the **Switches** subkey:

- SwitchxUp
- SwitchxUpRight
- SwitchxRight
- SwitchxDownRight
- SwitchxDown
- SwitchxDownLeft
- SwitchxUpLeft
- SwitchxLeft

The following table shows an example set of labels for switch positions of a 4-way switch which shows up at index 0 in the **RawGameController**:

Name	Value (type: REG_SZ)
Switch0Up	XboxUp
Switch0Right	XboxRight
Switch0Down	XboxDown
Switch0Left	XboxLeft

Example registry file

To show how all of these mappings and values come together, here is an example registry file for a generic **RacingWheel**:

text

Windows Registry Editor Version 5.00

```
[HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\GameInput\Devices\1234567800010004]
```

```
"Description" = "Example Wheel Device"
```

```
[HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\GameInput\Devices\1234567800010004\Labels\Buttons]
```

```
"Button0" = "LetterA"
```

```
"Button1" = "LetterB"
```

```
"Button2" = "LetterX"
```

```
"Button3" = "LetterY"
```

```
"Button6" = "Menu"
```

```
"Button7" = "View"
```

```
"Button8" = "RightStickButton"
```

```
"Button9" = "LeftStickButton"
```

```
[HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\GameInput\Devices\1234567800010004\Labels\Switches]
```

```
"Switch0Down" = "Down"  
"Switch0Left" = "Left"  
"Switch0Right" = "Right"  
"Switch0Up" = "Up"
```

```
[HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\GameInput\Devices\12345  
67800010004\RacingWheel]  
"MaxWheelAngle" = dword:000001c2
```

```
[HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\GameInput\Devices\12345  
67800010004\RacingWheel\Brake]  
"AxisIndex" = dword:00000002  
"Invert" = dword:00000001
```

```
[HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\GameInput\Devices\12345  
67800010004\RacingWheel\Button1]  
"ButtonIndex" = dword:00000000
```

```
[HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\GameInput\Devices\12345  
67800010004\RacingWheel\Button2]  
"ButtonIndex" = dword:00000001
```

```
[HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\GameInput\Devices\12345  
67800010004\RacingWheel\Button3]  
"ButtonIndex" = dword:00000002
```

```
[HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\GameInput\Devices\12345  
67800010004\RacingWheel\Button4]  
"ButtonIndex" = dword:00000003
```

```
[HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\GameInput\Devices\12345  
67800010004\RacingWheel\Button5]  
"ButtonIndex" = dword:00000009
```

```
[HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\GameInput\Devices\12345  
67800010004\RacingWheel\Button6]  
"ButtonIndex" = dword:00000008
```

```
[HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\GameInput\Devices\12345  
67800010004\RacingWheel\Button7]  
"ButtonIndex" = dword:00000007
```

```
[HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\GameInput\Devices\12345  
67800010004\RacingWheel\Button8]  
"ButtonIndex" = dword:00000006
```

```
[HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\GameInput\Devices\12345  
67800010004\RacingWheel\Clutch]  
"AxisIndex" = dword:00000003  
"Invert" = dword:00000001
```

```
[HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\GameInput\Devices\12345  
67800010004\RacingWheel\DPadDown]  
"IncludeAdjacent" = dword:00000001  
"SwitchIndex" = dword:00000000
```


"SwitchPosition" = "Down"

[HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\GameInput\Devices\1234567800010004\RacingWheel\DPadLeft]

"IncludeAdjacent" = dword:00000001

"SwitchIndex" = dword:00000000

"SwitchPosition" = "Left"

[HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\GameInput\Devices\1234567800010004\RacingWheel\DPadRight]

"IncludeAdjacent" = dword:00000001

"SwitchIndex" = dword:00000000

"SwitchPosition" = "Right"

[HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\GameInput\Devices\1234567800010004\RacingWheel\DPadUp]

"IncludeAdjacent" = dword:00000001

"SwitchIndex" = dword:00000000

"SwitchPosition" = "Up"

[HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\GameInput\Devices\1234567800010004\RacingWheel\FifthGear]

"ButtonIndex" = dword:00000010

[HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\GameInput\Devices\1234567800010004\RacingWheel\FirstGear]

"ButtonIndex" = dword:0000000c

[HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\GameInput\Devices\1234567800010004\RacingWheel\FourthGear]

"ButtonIndex" = dword:0000000f

[HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\GameInput\Devices\1234567800010004\RacingWheel\NextGear]

"ButtonIndex" = dword:00000004

[HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\GameInput\Devices\1234567800010004\RacingWheel\PreviousGear]

"ButtonIndex" = dword:00000005

[HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\GameInput\Devices\1234567800010004\RacingWheel\ReverseGear]

"ButtonIndex" = dword:0000000b

[HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\GameInput\Devices\1234567800010004\RacingWheel\SecondGear]

"ButtonIndex" = dword:0000000d

[HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\GameInput\Devices\1234567800010004\RacingWheel\SixthGear]

"ButtonIndex" = dword:00000011

[HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\GameInput\Devices\1234567800010004\RacingWheel\ThirdGear]

"ButtonIndex" = dword:0000000e

```
[HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\GameInput\Devices\1234567800010004\RacingWheel\Throttle]
"AxisIndex" = dword:00000001
"Invert" = dword:00000001

[HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\GameInput\Devices\1234567800010004\RacingWheel\Wheel]
"AxisIndex" = dword:00000000
"Invert" = dword:00000000
```

See also

- [Windows.Gaming.Input Namespace](#)
- [Windows.Gaming.Input.Custom Namespace](#)
- [INF Files](#)

Missing .NET APIs in Unity and UWP

Article • 03/16/2023

When building a UWP game using .NET, you may find that some APIs that you might use in the Unity editor or for a standalone PC game are not present for UWP. That's because .NET for UWP apps includes a subset of the types provided in the full .NET Framework for each namespace.

Additionally, some game engines use different flavors of .NET that aren't fully compatible with .NET for UWP, such as Unity's Mono. So when you're writing your game, everything might work fine in the editor, but when you go to build for UWP, you might get errors like this: **The type or namespace 'Formatters' does not exist in the namespace 'System.Runtime.Serialization' (are you missing an assembly reference?)**

Fortunately, Unity provides some of these missing APIs as extension methods and replacement types, which are described in [Universal Windows Platform: Missing .NET Types on .NET Scripting Backend](#) [↗](#). However, if the functionality you need is not here, [.NET for Windows 8.x apps overview](#) discusses ways you can convert your code to use WinRT or .NET for Windows Runtime APIs. (It discusses Windows 8, but is applicable to Windows 10 UWP apps as well.)

.NET Standard

To understand why some APIs might not be working, it's important to understand the different .NET flavors and how UWP implements .NET. The [.NET Standard](#) is a formal specification of .NET APIs that is meant to be cross-platform, and unify the different .NET flavors. Each implementation of .NET supports a certain version of the .NET Standard. You can see a table of standards and implementations at [.NET implementation support](#).

Each version of the UWP SDK conforms to a different level of .NET Standard. For example, the 16299 SDK (the Fall Creators Update) supports .NET Standard 2.0.

If you want to know if a certain .NET API is supported in the UWP version that you're targeting, you can check the [.NET Standard API Reference](#) and select the version of the .NET Standard that's supported by that version of UWP.

Scripting backend configuration

The first thing you should do if you're having trouble building for UWP is check the **Player Settings** (**File > Build Settings**, select **Universal Windows Platform**, and then **Player Settings**). Under **Other Settings > Configuration**, the first three dropdowns (**Scripting Runtime Version**, **Scripting Backend**, and **Api Compatibility Level**) are all important settings to consider.

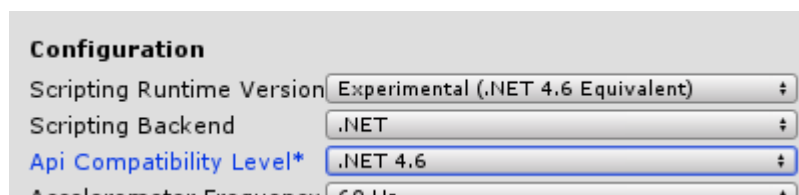
The **Scripting Runtime Version** is what the Unity scripting backend uses which allows you to get the (roughly) equivalent version of .NET Framework support that you choose. However, keep in mind that not all APIs in that version of the .NET Framework will be supported, only those in the version of .NET Standard that your UWP is targeting.

Often with new .NET releases, more APIs are added to .NET Standard which might allow you to use the same code across standalone and UWP. For example, the [System.Runtime.Serialization.Json](#) namespace was introduced in .NET Standard 2.0. If you set the **Scripting Runtime Version** to **.NET 3.5 Equivalent** (which targets an earlier version of the .NET Standard), you will get an error when trying to use the API; switch it to **.NET 4.6 Equivalent** (which supports .NET Standard 2.0), and the API will work.

The **Scripting Backend** can be **.NET** or **IL2CPP**. For this topic, we assume you have chosen **.NET**, since that's where the problems discussed here arise. See [Scripting Backends](#) for more information.

Finally, you should set the **Api Compatibility Level** to the version of .NET that you want your game to run on. This should match the **Scripting Runtime Version**.

In general, for **Scripting Runtime Version** and **Api Compatibility Level**, you should select the latest version available so as to have more compatibility with the .NET Framework, and thus allow you to use more .NET APIs.



Platform-dependent compilation

If you're building your Unity game for multiple platforms, including UWP, you'll want to use platform-dependent compilation to make sure that code intended for UWP is only run when the game is built as a UWP. This way, you can use the full .NET Framework for standalone desktop and other platforms, and WinRT APIs for UWP, without getting build errors.

Use the following directives to only compile code when running as a UWP app:

C#

```
#if NETFX_CORE
    // Your UWP code here
#else
    // Your standard code here
#endif
```

ⓘ Note

`NETFX_CORE` is only meant to check whether you're compiling C# code against the .NET scripting backend. If you're using a different scripting backend, such as IL2CPP, then use `ENABLE_WINMD_SUPPORT` instead.

Common issues and workarounds

The following scenarios describe common issues that might arise where .NET APIs are missing from the UWP subset, and ways to get around them.

Data serialization using `BinaryFormatter`

It is common for games to serialize save data so that players can't easily manipulate it. However, [BinaryFormatter](#), which serializes an object into binary, is not available in earlier versions of the .NET Standard (prior to 2.0). Consider using [XmlSerializer](#) or [DataContractJsonSerializer](#) instead.

C#

```
private void Save()
{
    SaveData data = new SaveData(); // User-defined object to serialize

    DataContractJsonSerializer serializer =
        new DataContractJsonSerializer(typeof(SaveData));

    FileStream stream =
        new FileStream(Application.persistentDataPath, FileMode.CreateNew);

    serializer.WriteObject(stream, data);
    stream.Dispose();
}
```

I/O operations

Some types in the [System.IO](#) namespace, such as [FileStream](#), are not available in earlier versions of the .NET Standard. However, Unity does provide the [Directory](#), [File](#), and [FileStream](#) types so you can use them in your game.

Alternatively, you can use the [Windows.Storage](#) APIs, which are only available to UWP apps. However, these APIs restrict the app to writing to their specific storage, and do not give it free access to the entire file system. See [Files, folders, and libraries](#) for more information.

One important note is that the [Close](#) method is only available in .NET Standard 2.0 and later (though Unity provides an extension method). Use [Dispose](#) instead.

Threading

Some types in the [System.Threading](#) namespaces, such as [ThreadPool](#), are not available in earlier versions of the .NET Standard. In these cases, you can use the [Windows.System.Threading](#) namespace instead.

Here's how you could handle threading in a Unity game, using platform-dependent compilation to prepare for both UWP and non-UWP platforms:

C#

```
private void UsingThreads()
{
    #if NETFX_CORE
        Windows.System.Threading.ThreadPool.RunAsync(workItem => SomeMethod());
    #else
        System.Threading.ThreadPool.QueueUserWorkItem(workItem => SomeMethod());
    #endif
}
```

Security

Some of the [System.Security.*](#) namespaces, such as [System.Security.Cryptography.X509Certificates](#), are not available when you build a Unity game for UWP. In these cases, use the [Windows.Security.*](#) APIs, which cover much of the same functionality.

The following example simply gets the certificates from a certificate store with the given name:

C#

```
private async void GetCertificatesAsync(string certStoreName)
{
    #if NETFX_CORE
        IReadOnlyList<Certificate> certs = await
CertificateStores.FindAllAsync();
        IEnumerable<Certificate> myCerts =
            certs.Where((certificate) => certificate.StoreName ==
certStoreName);
    #else
        X509Store store = new X509Store(certStoreName,
StoreLocation.CurrentUser);
        store.Open(OpenFlags.OpenExistingOnly);
        X509Certificate2Collection certs = store.Certificates;
    #endif
}
```

See [Security](#) for more information about using the WinRT security APIs.

Networking

Some of the **System.Net.*** namespaces, such as [System.Net.Mail](#), are also not available when building a Unity game for UWP. For most of these APIs, use the corresponding **Windows.Networking.*** and **Windows.Web.*** WinRT APIs to get similar functionality. See [Networking and web services](#) for more information.

In the case of **System.Net.Mail**, use the [Windows.ApplicationModel.Email](#) namespace. See [Send email](#) for more information.

See also

- [Universal Windows Platform: Missing .NET Types on .NET Scripting Backend](#) [↗](#)
- [.NET for UWP apps overview](#)

Networking for games

Article • 10/20/2022

Learn how to develop and incorporate networking features into your DirectX game.

Concepts at a glance

A variety of networking features can be used in your DirectX game, whether it is a simple standalone game to massively multi-player games. The simplest use of networking would be to store user names and game scores on a central network server.

Networking APIs are needed in multi-player games that use the infrastructure (client-server or internet peer-to-peer) model and also by ad hoc (local peer-to-peer) games. For server-based multi-player games, a central game server usually handles most of the game operations and the client game app is used for input, displaying graphics, playing audio, and other features. The speed and latency of network transfers is a concern for a satisfactory game experience.

For peer-to-peer games, each player's app handles the input and graphics. In most cases, the game players are located in close proximity so that network latency should be lower but is still a concern. How to discovery peers and establish a connection becomes a concern.

For single-player games, a central Web server or service is often used to store user names, game scores, and other miscellaneous information. In these games, the speed and latency of networking transfers is less of a concern since it doesn't directly affect game operation.

Network conditions can change at any time, so any game that uses networking APIs needs to handle network exceptions that may occur. To learn more about handling network exceptions, see [Networking basics](#).

Firewalls and web proxies are common and can affect the ability to use networking features. A game that uses networking needs to be prepared to properly handle firewalls and proxies.

For mobile devices, it is important to monitor available network resources and behave accordingly when on metered networks where roaming or data costs can be significant.

Network isolation is part of the app security model used by Windows. Windows actively discovers network boundaries and enforces network access restrictions for network isolation. Apps must declare network isolation capabilities in order to define the scope

of network access. Without declaring these capabilities, your app will not have access to network resources. To learn more about how Windows enforces network isolation for apps, see [How to configure network isolation capabilities](#).

Design considerations

A variety of networking APIs can be used in DirectX games. So, it is important to pick the right API. Windows supports a variety of networking APIs that your app can use to communicate with other computers and devices over either the Internet or private networks. Your first step is to figure out what networking features your app needs.

These are the more popular network APIs for games.

- TCP and sockets - Provides a reliable connection. Use TCP for game operations that don't need security. TCP allows the server to easily scale, so it is commonly used in games that use the infrastructure (client-server or internet peer-to-peer) model. TCP can also be used by ad hoc (local peer-to-peer) games over Wi-Fi Direct and Bluetooth. TCP is commonly used for game object movement, character interaction, text chat, and other operations. The [StreamSocket](#) class provides a TCP socket that can be used in Microsoft Store games. The **StreamSocket** class is used with related classes in the [Windows::Networking::Sockets](#) namespace.
- TCP and sockets using SSL - Provides a reliable connection that prevents eavesdropping. Use TCP connections with SSL for game operations that need security. The encryption and overhead of SSL adds a cost in latency and performance, so it is only used when security is needed. TCP with SSL is commonly used for login, purchasing and trading assets, game character creation and management. The [StreamSocket](#) class provides a TCP socket that supports SSL.
- UDP and sockets - Provides unreliable network transfers with low overhead. UDP is used for game operations that require low latency and can tolerate some packet loss. This is often used for fighting games, shooting and tracers, network audio, and voice chat. The [DatagramSocket](#) class provides a UDP socket that can be used in Microsoft Store games. The **DatagramSocket** class is used with related classes in the [Windows::Networking::Sockets](#) namespace.
- HTTP Client - Provides a reliable connection to HTTP servers. The most common networking scenario is to access a web site to retrieve or store information. A simple example would be a game that uses a website to store user information and game scores. When used with SSL for security, an HTTP client can be used for login, purchasing, trading assets, game character creation, and management. The [HttpClient](#) class provides a modern HTTP client API for use in Microsoft Store games. The **HttpClient** class is used with related classes in the [Windows::Web::Http](#) namespace.

Handling network exceptions in your DirectX game

When a network exception occurs in your DirectX game, this indicates a significant problem or failure. Exceptions can occur for many reasons when using networking APIs. Often, the exception can result from changes in network connectivity or other networking issues with the remote host or server.

Some causes of exceptions when using networking APIs include the following:

- Input from the user for a hostname or a URI contains errors and is not valid.
- Name resolutions failures when looking up a hostname or a URI.
- Loss or change in network connectivity.
- Network connection failures using sockets or the HTTP client APIs.
- Network server or remote endpoint errors.
- Miscellaneous networking errors.

Exceptions from network errors (for example, loss or change of connectivity, connection failures, and server failures) can happen at any time. These errors result in exceptions being thrown. If an exception is not handled by your app, it can cause your entire app to be terminated by the runtime.

You must write code to handle exceptions when you call most asynchronous network methods. Sometimes, when an exception occurs, a network method can be retried as a way to resolve the problem. Other times, your app may need to plan to continue without network connectivity using previously cached data.

Universal Windows Platform (UWP) apps generally throw a single exception. Your exception handler can retrieve more detailed information about the cause of the exception to better understand the failure and make appropriate decisions.

When an exception occurs in a DirectX game that is a UWP app, the **HRESULT** value for the cause of the error can be retrieved. The *Winerror.h* include file contains a large list of possible **HRESULT** values that includes network errors.

The networking APIs support different methods for retrieving this detailed information about the cause of an exception.

- A method to retrieve the **HRESULT** value of the error that caused the exception. The possible list of potential **HRESULT** values is large and unspecified. The **HRESULT** value can be retrieved when using any of the networking APIs.
- A helper method that converts the **HRESULT** value to an enumeration value. The list of possible enumeration values is specified and relatively small. A helper

method is available for the socket classes in the [Windows::Networking::Sockets](#).

Exceptions in Windows.Networking.Sockets

The constructor for the [HostName](#) class used with sockets can throw an exception if the string passed is not a valid hostname (contains characters that are not allowed in a host name). If an app gets input from the user for the **HostName** for a peer connection for gaming, the constructor should be in a try/catch block. If an exception is thrown, the app can notify the user and request a new hostname.

Add code to validate a string for a hostname from the user

C++/CX

```
// Define some variables at the class level.
Windows::Networking::HostName^ remoteHost;

bool isHostnameFromUser = false;
bool isHostnameValid = false;

///  

// If the value of 'remoteHostname' is set by the user in a control as input
// and is therefore untrusted input and could contain errors.
// If we can't create a valid hostname, we notify the user in statusText
// about the incorrect input.

String ^hostString = remoteHostname;

try
{
    remoteHost = ref new Windows::Networking::Host(hostString);
    isHostnameValid = true;
}
catch (InvalidArgumentException ^ex)
{
    statusText->Text = "You entered a bad hostname, please re-enter a valid  
hostname.";
    return;
}

isHostnameFromUser = true;

// ... Continue with code to execute with a valid hostname.
```

The [Windows.Networking.Sockets](#) namespace has convenient helper methods and enumerations for handling errors when using sockets. This can be useful for handling specific network exceptions differently in your app.

An error encountered on [DatagramSocket](#), [StreamSocket](#), or [StreamSocketListener](#) operation results in an exception being thrown. The cause of the exception is an error value represented as an **HRESULT** value. The [SocketError.GetStatus](#) method is used to convert a network error from a socket operation to a [SocketErrorStatus](#) enumeration value. Most of the **SocketErrorStatus** enumeration values correspond to an error returned by the native Windows sockets operation. An app can filter on specific **SocketErrorStatus** enumeration values to modify app behavior depending on the cause of the exception.

For parameter validation errors, an app can also use the **HRESULT** from the exception to learn more detailed information about the error that caused the exception. Possible **HRESULT** values are listed in the *Winerror.h* header file. For most parameter validation errors, the **HRESULT** returned is **E_INVALIDARG**.

Add code to handle exceptions when trying to make a stream socket connection

C++/CX

```
using namespace Windows::Networking;
using namespace Windows::Networking::Sockets;

// Define some more variables at the class level.

bool isConnected = false
bool retrySocketConnect = false;

// The number of times we have tried to connect the socket.
unsigned int retryConnectCount = 0;

// The maximum number of times to retry a connect operation.
unsigned int maxRetryConnectCount = 5;
///...

// We pass in a valid remoteHost and serviceName parameter.
// The hostname can contain a name or an IP address.
// The servicename can contain a string or a TCP port number.

StreamSocket ^ socket = ref new StreamSocket();
SocketErrorStatus errorStatus;
HRESULT hr;

// Save the socket, so any subsequent steps can use it.
CoreApplication::Properties->Insert("clientSocket", socket);

// Connect to the remote server.
create_task(socket->ConnectAsync(
    remoteHost,
    serviceName,
    SocketProtectionLevel::PlainSocket)).then([this] (task<void>
previousTask)
```

```

{
    try
    {
        // Try getting all exceptions from the continuation chain above
        this point.
        previousTask.get();

        isConnected = true;
        // Mark the socket as connected. We do not really care about the
        value of the property, but the mere
        // existence of it means that we are connected.
        CoreApplication::Properties->Insert("connected", nullptr);
    }
    catch (Exception^ ex)
    {
        hr = ex.HResult;
        errorStatus = SocketStatus::GetStatus(hr);
        if (errorStatus != Unknown)
        {
            switch
            (errorStatus)
            {
                case HostNotFound:
                    // If the hostname is from the user, this may
                    indicate a bad input.
                    // Set a flag to ask the user to re-enter the
                    hostname.

                    isHostnameValid = false;
                    return;
                    break;
                case ConnectionRefused:
                    // The server might be temporarily busy.
                    retrySocketConnect = true;
                    return;
                    break;
                case NetworkIsUnreachable:
                    // This could be a connectivity issue.
                    retrySocketConnect = true;
                    break;
                case UnreachableHost:
                    // This could be a connectivity issue.
                    retrySocketConnect = true;
                    break;
                case NetworkIsDown:
                    // This could be a connectivity issue.
                    retrySocketConnect = true;
                    break;
                // Handle other errors.
                default:
                    // The connection failed and no options are
                    available.

                    // Try to use cached data if it is available.
                    // You may want to tell the user that the connect
                    failed.

                    break;
            }
        }
    }
}

```

```

    }
    }
    else
    {
        // Received an HRESULT that is not mapped to an enum.
        // This could be a connectivity issue.
        retrySocketConnect = true;
    }
}
});
}

```

Exceptions in Windows.Web.Http

The constructor for the [Windows::Foundation::Uri](#) class used with [Windows::Web::Http::HttpClient](#) can throw an exception if the string passed is not a valid URI (contains characters that are not allowed in a URI). In C++, there is no method to try and parse a string to a URI. If an app gets input from the user for the [Windows::Foundation::Uri](#), the constructor should be in a try/catch block. If an exception is thrown, the app can notify the user and request a new URI.

Your app should also check that the scheme in the URI is HTTP or HTTPS since these are the only schemes supported by the [Windows::Web::Http::HttpClient](#).

Add code to validate a string for a URI from the user

C++/CX

```

// Define some variables at the class level.
Windows::Foundation::Uri^ resourceUri;

bool isUriFromUser = false;
bool isUriValid = false;

///...

// If the value of 'inputUri' is set by the user in a control as input
// and is therefore untrusted input and could contain errors.
// If we can't create a valid hostname, we notify the user in statusText
// about the incorrect input.

String ^uriString = inputUri;

try
{
    isUriValid = false;
    resourceUri = ref new Windows::Foundation::Uri(uriString);
}

```

```

        if (resourceUri->SchemeName != "http" && resourceUri->SchemeName !=
"https")
        {
            statusText->Text = "Only 'http' and 'https' schemes supported.
Please re-enter URI";
            return;
        }
        isUriValid = true;
    }
    catch (InvalidArgumentException ^ex)
    {
        statusText->Text = "You entered a bad URI, please re-enter Uri to
continue.";
        return;
    }

    isUriFromUser = true;

    // ... Continue with code to execute with a valid URI.

```

The [Windows::Web::Http](#) namespace lacks a convenience function. So, an app using [HttpClient](#) and other classes in this namespace needs to use the **HRESULT** value.

In apps using C++, the [Platform::Exception](#) represents an error during app execution when an exception occurs. The [Platform::Exception::HResult](#) property returns the **HRESULT** assigned to the specific exception. The [Platform::Exception::Message](#) property returns the system-provided string that is associated with the **HRESULT** value. Possible **HRESULT** values are listed in the *Winerror.h* header file. An app can filter on specific **HRESULT** values to modify app behavior depending on the cause of the exception.

For most parameter validation errors, the **HRESULT** returned is **E_INVALIDARG**. For some illegal method calls, the **HRESULT** returned is **E_ILLEGAL_METHOD_CALL**.

Add code to handle exceptions when trying to use [HttpClient](#) to connect to an HTTP server

C++/CX

```

using namespace Windows::Foundation;
using namespace Windows::Web::Http;

// Define some more variables at the class level.

bool isHttpClientConnected = false
bool retryHttpClient = false;

// The number of times we have tried to connect the socket
unsigned int retryConnectCount = 0;

```

```

// The maximum number of times to retry a connect operation.
unsigned int maxRetryConnectCount = 5;
///  

// We pass in a valid resourceUri parameter.
// The URI must contain a scheme and a name or an IP address.

HttpClient ^ httpClient = ref new HttpClient();
HRESULT hr;

// Save the httpClient, so any subsequent steps can use it.
CoreApplication::Properties->Insert("httpClient", httpClient);

// Send a GET request to the HTTP server.
create_task(httpClient->GetAsync(resourceUri)).then([this] (task<void>
previousTask)
{
    try
    {
        // Try getting all exceptions from the continuation chain above
this point.
        previousTask.get();

        isHttpClientConnected = true;
        // Mark the HttpClient as connected. We do not really care about
the value of the property, but the mere
        // existence of it means that we are connected.
        CoreApplication::Properties->Insert("connected", nullptr);
    }
    catch (Exception^ ex)
    {
        hr = ex.HResult;

        switch (errorStatus)
        {
            case WININET_E_NAME_NOT_RESOLVED:
                // If the Uri is from the user, this may indicate a bad
input.

                // Set a flag to ask user to re-enter the Uri.
                isUriValid = false;
                return;
                break;
            case WININET_E_CANNOT_CONNECT:
                // The server might be temporarily busy.
                retryHttpClientConnect = true;
                return;
                break;
            case WININET_E_CONNECTION_ABORTED:
                // This could be a connectivity issue.
                retryHttpClientConnect = true;
                break;
            case WININET_E_CONNECTION_RESET:
                // This could be a connectivity issue.
                retryHttpClientConnect = true;
                break;
            case INET_E_RESOURCE_NOT_FOUND:

```



```

        // The server cannot locate the resource specified in
the uri.

        // If the Uri is from user, this may indicate a bad
input.

        // Set a flag to ask the user to re-enter the Uri
isUriValid = false;
        return;
        break;
    // Handle other errors.
    default:
        // The connection failed and no options are available.
        // Try to use cached data if it is available.
        // You may want to tell the user that the connect
failed.

        break;
    }
    else
    {
        // Received an HRESULT that is not mapped to an enum.
        // This could be a connectivity issue.
        retrySocketConnect = true;
    }
}
});

```

Related topics

Other resources

- [Connecting with a datagram socket](#)
- [Connecting to a network resource with a stream socket](#)
- [Connecting to network services](#)
- [Connecting to web services](#)
- [Networking basics](#)
- [How to configure network isolation capabilities](#)
- [How to enable loopback and debug network isolation](#)

Reference

- [DatagramSocket](#)
- [HttpClient](#)
- [StreamSocket](#)
- [Windows::Web::Http namespace](#)
- [Windows::Networking::Sockets namespace](#)

Sample apps

- [DatagramSocket sample](#)
- [HttpClient sample](#) ↗
- [Proximity sample](#) ↗
- [StreamSocket sample](#)

Game development videos

Article • 10/20/2022

Here's a list of game development videos from major conferences like GDC and //build.

GDC 2018 videos

PlayFab

What is LiveOps? Learn why developers are creating live games: [Your game for everyone with PlayFab \(Interview\)](#) - James Gwertzman, David Holladay

- [Fluffy Fairy's Lean Approach to Game Development - How an Indie studio grew a hit game after only 8 weeks](#) - Oliver Löffler from Fluffy Fairy
- [Day 1: Your game, everywhere](#) - Brendan Vanous
- [Day 2: Your game for everyone with PlayFab \(Presentation\)](#) - Brendan Vanous
- [Day 3: Your game, every way it's played](#) - Brendan Vanous

Azure cloud services

- [How The Walking Dead No Man's Land game was built to scale on Azure](#) - Kalle Hiitola from Next Games
- [Azure Cosmos DB: Comprehensive overview](#) - Aleksey Savateyev
- [Containers, Clusters and the Cloud for Gaming](#) - James Trott
- [Rendering in Azure](#) - John Thuotte
- [Running on a Hyper-V](#) - Andrew Farrier

Xbox Live Creators Program

- [Xbox Live Creators Program jumpstart for Unity developers](#) - Ramsey Khadder

Mixer

- [Get massive promotion for minimal effort with Mixer](#) - Gersh Payzer

Visual Studio Team Services (VSTS) and App Center

What is VSTS? Learn how to implement DevOps in your build process: [Introduction to Visual Studio Team Services](#) - Dmitry Lyalin, James Montemagno

- [Planning and building games using the full power of VSTS: Agile, CI & CD end-to-end demo](#) - Rogan Ferguson
- [Build your next game powered by Visual Studio Team Services and Unity Teams cloud build](#) - Dmitry Lyalin
- [Continuously test, distribute and monitor your game with App Center](#) - James White
- [Setup your CI/CD pipeline for Node.js, Python, .NET, Go, Ruby, or Java - in minutes](#) - Abel Wang
- [Agile tooling set with VSTS](#) - Rogan Ferguson, James Montemagno

Git

New to version control systems? Intro to Git and Git Virtual File System (GVFS):
[Introduction to GVFS](#) - Edward Thomson, James Montemagno

- [Working with large binary game assets using Git Large File system](#) - Edward Thomson
- [Git with Unity for game development](#) - Edward Thomson

Visual Studio

- [Unity and Visual Studio, better together](#) - Arturo Nunez from Unity

Game development experiences

- [Photogrammetry and laser scanning in Forza Motorsport 7](#) - Travis Gosnell
- [Building Astroneer: Charting new and challenging courses](#) - Brendan Wilson

Microsoft Store

- [Microsoft Store: How you can reach Xbox One and Windows 10 users](#) - Bernardo Zamora
- [Why Candy Crush on Windows 10 and in Microsoft Store](#) - Alex von Roth from King

Mixed Reality

- [Building Windows MR Apps with Unity](#) - Jaime Rodriguez
- [The AR Cloud: Why the future of MR is not a device, it's all of them](#) - Jesse McCulloch

Game Accessibility Conference 2018

- [Looking Bright. 2017's Game Accessibility Advances](#) - Ian Hamilton, Accessibility industry expert
- [The CVAA and What it Means for Gaming Access](#) - Karen Petlz Strauss from the Federal Communications Commission
- [I Made My Game Blind Accessible – You Won't Believe What Happened Next...](#) - Michelle Martin from MetalPop Games
- [Cognitive Accessibility Design Choices and Music Education for Autism](#) - Lori Landay from the Berklee College of Music
- [Ramping up accessibility](#) - Karen Stevens from EA Sports
- [Modern Game Accessibility. Changing Sympathy to Solution](#) - Kevin Keeker from Playstation and Sam Thompson from Sony World Wide Studios
- [Playing Mainstream Video Games Without Sight](#) - SightlessKombat, Accessibility advocate
- [How Community Collaboration Improves Development](#) - Jason Canam from HouseHold Games
- [How Live Streaming Brings Next Level Inclusion](#) - Tara Voelker from Mixer
- [All Deaf Gamers Wanna Do is Have Some Fun](#) - Panel speakers: Chris Robinson and Kristen Piersa, Accessibility advocates. Michael Anthony from Microsoft
- [A Fraught Love Letter to the Games Industry](#) - Cherry Thompson, Accessibility advocate
- [Accessible Gaming History](#) - Barrie Ellis from OneSwitch and Special Effect

//Build 2017 videos

- [Lighting up HDR and advanced color in DirectX](#) - Simon Tao

GDC 2017 game dev session videos

- [Unveiling New Game Developer Opportunities with Xbox and Windows](#) - Chris Charla
- [Introducing Spatial Audio Capabilities for Xbox and Windows](#) - Scott Selfon
- [D3D12 Performance Tuning and Debugging with PIX and GPU Validation](#) - Shawn Hargreaves, Max McMullen
- [Game Mode and the Evolution of Cross-Platform UWP Game Development](#) - Eric Walston
- [Xbox Live for All Game](#) - Brian Tyler
- [Meet the Converged Microsoft Store for Windows and Xbox](#) - Krupa Poobala-Chandran

- Windows Holographic Rendering: One SDK to Target VR and AR Ecosystems - Alex Pfaffe, Alex Turner
- Reaching More Gamers Through Xbox Accessibility Features on Xbox One and PC - Evelyn Thomas

GDC 2015 game dev session videos

- The Future of Gaming Across the Microsoft Ecosystem – Phil Spencer
- Developing Games for Windows 10 – Chris Tector, Don Box
- Advanced DirectX 12 Graphics and Performance – Max McMullen
- Gaming Consumer Experience on Windows 10 – Bill Schiefelbein
- New Opportunities for Independent Developers – Chris Charla
- Developing with Xbox Live for Windows 10 – Vijay Gajjala, Brian Tyler
- Xbox Live Multiplayer: Introducing services for cross-platform matchmaking and gameplay – Ferdinand Schober
- Fable Legends: Cross-device Gameplay with Xbox Live – Raymond Arifianto, Julian Davis
- Best Practices for Leveraging Cloud-Based User Stats and Achievements in Xbox Live – Cierra McDonald
- Solve the Tough Graphics Problems with Your Game Using DirectX Tools – Rudolph Balaz, Rong Lu, Jason Strayer
- Better Power, Better Performance: Your Game on DirectX 12 – Bennett Sorbo
- Sustained Gaming Performance in multi-core mobile devices – Julian Calinov
- Designing Games for a Windows Core World – Todd Williams, Shai Hinitz
- PC Games for Windows 10 – Chuck Walbourn

//build 2015 session videos

- Introducing Win2D: DirectX-Powered Drawing in C# – Simon Tao
- ANGLE: Running OpenGL ES 2.0 Graphics Code on Windows – Tony Balogh
- Advanced DirectX12 Graphics and Performance – Max McMullen

Archive

These are videos that were in the Windows game development guide.

- Accelerating Microsoft Store Game Development with Middleware
 - Game Development Middleware - What is it? Do I need it?
-

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

DirectX programming

Article • 10/20/2022

This section provides information about developing UWP games with DirectX.

Topic	Description
Getting started	Introduction to DirectX programming.
Samples	Learn DirectX through game samples.
Fundamentals	Explains DirectX basic programming concepts.
Add features	Describes how to add various game features into your DirectX game.
Optimization and advanced topics	Learn about optimization and other advanced topics.
Port DirectX 9 to UWP	Learn how to port your DirectX 9 game to UWP.
Port OpenGL to Direct3D	Learn how to port your OpenGL ES 2.0 game to Direct3D 11.

For DirectX reference and programming guides, see [Direct3D 11 Graphics](#) and [Direct3D 12 Graphics](#).

DirectX: Getting started

Article • 10/20/2022

This section provides information to help you get started on developing UWP games using DirectX.

The [app object and DirectX](#) topic explains how to use the core user interface framework with DirectX. Since DirectX games run at a lower level in the Windows Runtime stack, they need to interoperate with the user interface framework more fundamentally: by accessing and interoperating with the app object directly.

The [Project templates and tools for games](#) topic explains how to create a DirectX game project from a template. It introduces editing and diagnostic tools like Model Editor, Shader Designer, and Graphics Frame Analysis which can help you to create, preview, and troubleshoot DirectX graphics.

The [Launching and resuming apps](#) topic explains how you can define the activation experience of a UWP DirectX game, how to save important system state and app data, and restore important application data when your game resumes.

Topic	Description
The app object and DirectX	Access and interoperate with the app object directly.
Project templates and tools for games	Prepare your dev environment for UWP DirectX game development.
Launching and resuming apps	Launch, suspend, and resume your UWP DirectX game.

The app object and DirectX

Article • 10/20/2022

Universal Windows Platform (UWP) with DirectX games don't use many of the Windows UI user interface elements and objects. Rather, because they run at a lower level in the Windows Runtime stack, they must interoperate with the user interface framework in a more fundamental way: by accessing and interoperating with the app object directly. Learn when and how this interoperation occurs, and how you, as a DirectX developer, can effectively use this model in the development of your UWP app.

See the [Direct3D graphics glossary](#) for information about unfamiliar graphics terms or concepts you encounter while reading.

The important core user interface namespaces

First, let's note the Windows Runtime namespaces that you must include (with **using**) in your UWP app. We get into the details in a bit.

- [Windows.ApplicationModel.Core](#)
- [Windows.ApplicationModel.Activation](#)
- [Windows.UI.Core](#)
- [Windows.System](#)
- [Windows.Foundation](#)

ⓘ Note

If you're not developing a UWP app, use the user interface components provided in the JavaScript- or XAML-specific libraries and namespaces instead of the types provided in these namespaces.

The Windows Runtime app object

In your UWP app, you want to get a window and a view provider from which you can get a view and to which you can connect your swap chain (your display buffers). You can also hook this view into the window-specific events for your running app. To get the parent window for the app object, defined by the [CoreWindow](#) type, create a type that implements [IFrameworkViewSource](#). For a [C++/WinRT](#) code example showing how to implement [IFrameworkViewSource](#), see [Composition native interoperation with DirectX and Direct2D](#).

Here's the basic set of steps to get a window using the core user interface framework.

1. Create a type that implements [IFrameworkView](#). This is your view.

In this type, define:

- An [Initialize](#) method that takes an instance of [CoreApplicationView](#) as a parameter. You can get an instance of this type by calling [CoreApplication.CreateNewView](#). The app object calls it when the app is launched.
- A [SetWindow](#) method that takes an instance of [CoreWindow](#) as a parameter. You can get an instance of this type by accessing the [CoreWindow](#) property on your new [CoreApplicationView](#) instance.
- A [Load](#) method that takes a string for an entry point as the sole parameter. The app object provides the entry point string when you call this method. This is where you set up resources. You create your device resources here. The app object calls it when the app is launched.
- A [Run](#) method that activates the [CoreWindow](#) object and starts the window event dispatcher. The app object calls it when the app's process starts.
- An [Uninitialize](#) method that cleans up the resources set up in the call to [Load](#). The app object calls this method when the app is closed.

2. Create a type that implements [IFrameworkViewSource](#). This is your view provider.

In this type, define:

- A method named [CreateView](#) that returns an instance of your [IFrameworkView](#) implementation, as created in Step 1.

3. Pass an instance of the view provider to [CoreApplication.Run](#) from **main**.

With those basics in mind, let's look at more options you have to extend this approach.

Core user interface types

Here are other core user interface types in the Windows Runtime that you might find helpful:

- [Windows.ApplicationModel.CoreCoreApplicationView](#)
- [Windows.UI.Core.CoreWindow](#)
- [Windows.UI.Core.CoreDispatcher](#)

You can use these types to access your app's view, specifically, the bits that draw the contents of the app's parent window, and handle the events fired for that window. The

app window's process is an *application single-threaded apartment* (ASTA) that is isolated and that handles all callbacks.

Your app's view is generated by the view provider for your app window, and in most cases will be implemented by a specific framework package or the system itself, so you don't need to implement it yourself. For DirectX, you need to implement a thin view provider, as discussed previously. There is a specific 1-to-1 relationship between the following components and behaviors:

- An app's view, which is represented by the [CoreApplicationView](#) type, and which defines the method(s) for updating the window.
- An ASTA, the attribution of which defines the threading behavior of the app. You cannot create instances of COM STA-attributed types on an ASTA.
- A view provider, which your app obtains from the system or which you implement.
- A parent window, which is represented by the [CoreWindow](#) type.
- Sourcing for all activation events. Both views and windows have separate activation events.

In summary, the app object provides a view provider factory. It creates a view provider and instantiates a parent window for the app. The view provider defines the app's view for the parent window of the app. Now, let's discuss the specifics of the view and the parent window.

CoreApplicationView behaviors and properties

[CoreApplicationView](#) represents the current app view. The app singleton creates the app view during initialization, but the view remains dormant until it is activated. You can get the [CoreWindow](#) that displays the view by accessing the [CoreApplicationView.CoreWindow](#) property on it, and you can handle activation and deactivation events for the view by registering delegates with the [CoreApplicationView.Activated](#) event.

CoreWindow behaviors and properties

The parent window, which is a [CoreWindow](#) instance, is created and passed to the view provider when the app object initializes. If the app has a window to display, it displays it; otherwise, it simply initializes the view.

[CoreWindow](#) provides a number of events specific to input and basic window behaviors. You can handle these events by registering your own delegates with them.

You can also obtain the window event dispatcher for the window by accessing the [CoreWindow.Dispatcher](#) property, which provides an instance of [CoreDispatcher](#).

CoreDispatcher behaviors and properties

You can determine the threading behavior of event dispatching for a window with the [CoreDispatcher](#) type. On this type, there's one particularly important method: the [CoreDispatcher.ProcessEvents](#) method, which starts window event processing. Calling this method with the wrong option for your app can lead to all sorts of unexpected event processing behaviors.

CoreProcessEventsOption option	Description
CoreProcessEventsOption.ProcessOneAndAllPending	Dispatch all currently available events in the queue. If no events are pending, wait for the next new event.
CoreProcessEventsOption.ProcessOnelfPresent	Dispatch one event if it is pending in the queue. If no events are pending, don't wait for a new event to be raised but instead return immediately.
CoreProcessEventsOption.ProcessUntilQuit	Wait for new events and dispatch all available events. Continue this behavior until the window is closed or the application calls the Close method on the CoreWindow instance.
CoreProcessEventsOption.ProcessAllIfPresent	Dispatch all currently available events in the queue. If no events are pending, return immediately.

UWP using DirectX should use the [CoreProcessEventsOption.ProcessAllIfPresent](#) option to prevent blocking behaviors that might interrupt graphics updates.

ASTA considerations for DirectX devs

The app object that defines the run-time representation of your UWP and DirectX app uses a threading model called Application Single-Threaded Apartment (ASTA) to host your app's UI views. If you are developing a UWP and DirectX app, you're familiar with the properties of an ASTA, because any thread you dispatch from your UWP and DirectX app must use the [Windows::System::Threading](#) APIs, or use [CoreWindow::CoreDispatcher](#). (You can get the [CoreWindow](#) object for the ASTA by calling [CoreWindow::GetForCurrentThread](#) from your app.)